ISIT307 - WEB SERVER PROGRAMMING

LECTURE 2 – MANIPULATING STRINGS &

HANDLING USER INPUT

LECTURE PLAN

- String manipulation
- String parsing
- Regular expressions
- Autoglobal variables
- Web forms
- Processing form data
- Create an All-in-One form
- Display dynamic content

TEXT STRINGS

- A text string contains zero or more characters surrounded by double or single quotation marks
- Text strings can be used as literal values or assigned to a variable

```
echo "<p>PHP literal text string</p>";
$stringVariable = "<p>PHP literal text string</p>";
echo $stringVariable;
```

- A string must begin and end with a matching quotation mark (single or double)

ADDING ESCAPE CHARACTERS AND SEQUENCES

- An **escape character** tells the compiler or interpreter that the character that follows it has a special purpose
- In PHP, the escape character is the backslash (\)

```
echo '<p>This code\'s going to work</p>';

echo "<p>\"Be ready\" they said.</p>";
```

- Do not add a backslash before an apostrophe if you surround the text string with double quotation marks

```
echo "<p>This code's going to work</p>";

echo '<p>"Be ready" they said.</p>';
```

ADDING ESCAPE CHARACTERS AND SEQUENCES

- The escape character combined with one or more other characters is an **escape sequence**

Sequence	Meaning
\n	linefeed
\r	carriage return
\t	horizontal tab
\v	vertical tab
\e	escape
\f	form feed
\\	backslash
\$	dollar sign
\"	double-quote

**PHP escape sequences within double quotation marks*

COMPLEX STRING SYNTAX

- Variables can be placed within curly braces inside of a string, it is called **complex string syntax**

```
$many = "page";
echo "<p>How many {$many}s you have?</p>";
```

COUNTING CHARACTERS AND WORDS IN A STRING

- `strlen()` function is used to get the length of a string (the number of bytes)
- `str_word_count()` function returns the number of words in a string

```
$title = "I love PHP";
echo "<p>The title contains " . strlen($title) . " chars, ";
echo " and " . str_word_count($title) . " words.</p>";
```

CASE CONVERSIONS

- PHP provides several functions to convert strings to uppercase and lowercase
  - `strtoupper()` - converts all letters in a string to uppercase
  - `strtolower()` - converts all letters in a string to lowercase
  - `ucfirst()` - converts first character of a string to uppercase
  - `lcfirst()` - converts first character of a string to lowercase
  - `ucwords()` - converts the first character of each word to uppercase

HTMLSPECIALCHARS()

- Some characters in HTML have a special meaning
- Functions that can be used to handle strings that may contain HTML tags
  - `htmlspecialchars()` - converts special characters to HTML entities
  - The `html_specialcharacters_decode()` - converts HTML character entities into their equivalent characters
    - '&' (ampersand) becomes '&'
    - '"' (double quote) becomes '"'
    - ''' (single quote) becomes '''
    - '<' (less than) becomes '<'
    - '>' (greater than) becomes '>'

REMOVE LEADING/TRAILING SPACES

- To remove leading or trailing spaces in a string the following functions can be used:
  - `trim()` - removes leading or trailing spaces in a string
  - `ltrim()` - removes only the leading spaces
  - `rtrim()` - removes only the trailing spaces

STRING MANIPULATION

- `substr()` - returns part of a string based on the values of the `start` and `length` parameters

```
substr(string, start, optional length);
```

- `start` parameter
  - positive value indicates how many character to skip at the beginning of the string
  - negative value indicates how many characters to count in from the end of the string
- `length` parameter
  - positive value determines how many characters to return
  - negative value defines skipping that many characters at the end of the string and returns the middle portion
  - If the length is omitted or is greater than the remaining length of the string, the entire remainder of the string is returned

OTHER WAYS TO MANIPULATE A STRING - EXAMPLE

```
  $ExampleString = "woodworking project";
  echo substr($ExampleString,4) . "<br>";
  echo substr($ExampleString,4,7) . "<br>";
  echo substr($ExampleString,0,8) . "<br>";
  echo substr($ExampleString,-7) . "<br>";
  echo substr($ExampleString,-12,4) . "<br>";
  echo substr($ExampleString,5,-2) . "<br>";
---
  echo strrev($ExampleString) . "<br>";
  echo str_shuffle($ExampleString) . "<br>";
```

STRING MANIPULATION

- `strstr()` – search and return a substring from the specified substring
- `str_replace(), str_ireplace()` - replace the substring within the string
  - 1st argument: the string to search for
  - 2nd argument: the replacement string
  - 3rd argument: the string in which characters are replaced

STRING SEARCH

- `strpos()` - performs a case-sensitive search and returns the position of the first occurrence of one string in another string
  - The first argument is the string you want to search
  - The second argument contains the characters for which you want to search
  - If the search string is not found, the `strpos()` function returns a Boolean value of `FALSE`

FINDING, EXTRACTING, REPLACING CHARACTERS AND SUBSTRINGS

```
<?php
      $Email = "my.email@uow.edu.au";
      echo "<p>if I use strstr - " . strstr($Email, ".") . "</p>";
      echo "<p>if I use strstr - " . strstr($Email, ".ed") . "</p>";

      echo "<p> the @ is at position - " . strpos($Email, "@") . "</p>";

      echo "<p>if I replace the email - " .
                        str_replace("email", "e-mail", $Email) . "</p>";

      if (strpos($Email, "m") === FALSE)
            echo "the char is not found";
      else
            echo "the char is at position ", strpos($Email, "m");
?>
```

STRING PARSING

- `str_split()` - splits each character of a string into an array element

```
$array = str_split(string[,length]);
```

  - *length* argument defines the number of characters to be assigned to each array element

- `explode()` - splits a string into an indexed array at a specified separator

```
$array = explode(separators, string);
```

STRING PARSING

```
$subjects = "CSIT128; CSIT884; CSIT323; MTS9307";
$subjectsArray = explode(";", $subjects);
foreach ($subjectsArray as $subject) {
        echo "$subject <br />";
}
```

- If the string does not contain the specified separator, the entire string is assigned to the first element of the array

STRING PARSING

- `implode()` - combines an array's elements into a single string, separated by specified characters

```
$variable = implode(separators, array);
```

```
$subjectsArray = array("CSIT128","CSIT884","CSIT323,"MTS9307");
$subjects = implode(", ", $subjectsArray);
echo $subjects;
```

STRING COMPARISON

- `strcasecmp()` - performs a case-insensitive comparison of strings
- `strcmp()` - performs a case-sensitive comparison of strings
- The functions return
  - negative value if the first string is less than the second (appears before alphabetically)
  - zero if the two strings are equal
  - positive value if the first string is greater than the second

STRING COMPARISON

- `str_contains()` - checks whether a string contains another string
- `str_starts_with()` - checks whether a string starts with another string
- `str_ends_with()` - checks whether a string ends with another string
- All 3 functions return Boolean value true/false

EXAMPLE – WHAT IS THE OUTPUT?

```
<?php
$my_str = "Bob is working";
echo "<p>$my_str</p>";

$my_str[0] = "R";
echo "<p>$my_str</p>";

for($i=0; $i< strlen($my_str); $i++)
     echo "<p>$my_str[$i]</p>";
?>
```

REGULAR EXPRESSIONS

- **Regular Expressions** are patterns that are used for matching and manipulating strings according to specified rules
- Most commonly these expressions are used to ensure that the user has entered the data in correct format
- PHP supports two types of regular expressions:
  - POSIX Extended
  - Perl Compatible Regular Expressions (PCRE)

REGULAR EXPRESSIONS

- Pass to the `preg_match()` the regular expression pattern as the first argument and a string containing the text you want to search as the second argument

```
preg_match(pattern, string);
```

- The function returns 1 if the specified pattern is matched or a value of 0 if it is not matched
- The pattern is case-sensitive by default, and need to use following syntax for case-insensitive matching

```
preg_match("/pattern/i", string);
```

REGULAR EXPRESSION PATTERNS

- A **regular expression pattern** is a special text string that describes a search pattern
- Regular expression patterns consist of literal characters and **metacharacters**, which are special characters that define the pattern-matching rules
- Regular expression patterns are enclosed in opening and closing **delimiters**
  - The most common character delimiter is the forward slash (/)

REGULAR EXPRESSION PATTERNS - METACHARACTERS

Metacharacter	Description
.	Matches any single character
\	Identifies the next character as a literal value
^	Anchors characters to the beginning of a string
$	Anchors characters to the end of a string
()	Specifies required characters to include in a pattern match
[]	Specifies alternate characters allowed in a pattern match
[^]	Specifies characters to exclude in a pattern match
-	Identifies a possible range of characters to match
|	Specifies alternate sets of characters to include in a pattern match

PHP Programming with MySQL, 2011, Cengage Learning.

MATCHING ANY CHARACTER

- A period (`.`) in a regular expression pattern specifies that the pattern must contain a value at the location of the period
- A return value of `0` indicates that the string does not match the pattern and `1` if it does

```
$ZIP = "015";
preg_match("/...../", $ZIP); // returns 0

$ZIP = "01562";
preg_match("/...../", $ZIP); // returns 1
```

MATCHING CHARACTERS AT THE BEGINNING OR END OF A STRING

- An **anchor** specifies that the pattern must appear at a particular position in a string
- The ^ metacharacter anchors characters to the beginning of a string
- The $ metacharacter anchors characters to the end of a string

```
$URL = "http://www.education.com";
preg_match("/^http/", $URL); // returns 1
```

```
$URL = "http://www.education.com";
preg_match("/com$/", $URL); // returns 1
```

MATCHING SPECIAL CHARACTERS

- To match any metacharacters as literal values in a regular expression, escape the character with a backslash
- For example if we want to ensure that the string contains actual period, can be used

```
$Identifier = "http://www.education.com";

preg_match("/\.com$/", $Identifier);//returns 1
```

MATCHING SPECIAL CHARACTERS

- With some metacharacters the escape sequence is more complicated, so single quotes instead can be used
- For example if we want to ensure that the string contains dollar sign, can be used

```
$Identifier = "$1234.56";

preg_match('/^\$/', $Identifier);//returns 1

preg_match("/^\\\$/", $Identifier);//returns ?
```

SPECIFYING QUANTITY

- Metacharacters that specify the quantity of a match are called **quantifiers**

Quantifier	Description
?	Specifies that the preceding character is optional
+	Specifies that one or more of the preceding characters must match
*	Specifies that zero or more of the preceding characters can match
{*n*}	Specifies that the preceding character repeat exactly *n* times
{*n*,}	Specifies that the preceding character repeat at least *n* times
{,*n*}	Specifies that the preceding character repeat up to *n* times
{*n1*, *n2*}	Specifies that the preceding character repeat at least *n1* times but no more than *n2* times

SPECIFYING QUANTITY

- A question mark `(?)` quantifier specifies that the preceding character in the pattern is optional (in the following example, the string must begin with 'http' or 'https')

```
$URL = "http://www.education.com";
preg_match("/^https?/", $URL); // returns 1
```

SPECIFYING QUANTITY

- The addition (+) quantifier specifies that one or more sequential occurrences of the preceding characters match

  (in the following example, the string must have at least one character)

```
$Name = "Don";
preg_match("/.+/", $Name); // returns 1
```

SPECIFYING QUANTITY

- A asterisk (*) quantifier specifies that zero or more sequential occurrences of the preceding characters match
  (in the following examples, the string might begin with one or more leading zeros)

```
NumberString = "00125";
preg_match("/^0*/", $NumberString); //returns 1
---
NumberString = "1234056";
preg_match("/^0*/", $NumberString); //returns 1
```

SPECIFYING QUANTITY

- The { } quantifiers specify the number of times that a character must repeat sequentially

(in the following example, the string must contain "ZIP:" plus five characters)

```
preg_match("/ZIP:.{5}$/", " ZIP:01562");
                                  // returns 1
```

- The { } quantifiers can also specify the quantity as a range

(in the following example, the string must contain "ZIP:" plus between five and ten characters)

```
preg_match("/(ZIP:.{5,10})$/", "ZIP:01562-2607");
                                      // returns 1
```

SPECIFYING SUBEXPRESSIONS

- A set of characters enclosed in parentheses are treated as a group -they are referred to as a **subexpression** or **subpattern**
  (in the example below, the 1 and the (nnn) are optional, but if included must be in the following format "1 (nnn)nnn-nnnn")

```
preg_match("/^(1 )?(\(.{3}\))?(.{3})(\-.{4})$/",
                                        "555-1234"); //return 1
preg_match("/^(1 )?(\(.{3}\))?(.{3})(\-.{4})$/",
                                    "(707)555-1234"); //return 1
preg_match("/^(1 )?(\(.{3}\))?(.{3})(\-.{4})$/",
                                  "1 (707)555-1234"); //return 1
```

DEFINING CHARACTER CLASSES

- **Character classes** in regular expressions treat multiple characters as a single item
- Characters enclosed with the ([]) metacharacters represent alternate characters that are allowed in a pattern match

```
preg_match("/analy[sz]e/", "analyse");//returns 1

preg_match("/analy[sz]e/", "analyze");//returns 1

preg_match("/analy[sz]e/", "analyce");//returns 0
```

DEFINING CHARACTER CLASSES

- The hyphen metacharacter (-) specifies a range of values in a character class
  (the following example ensures that A, B, C, D, or F are the only values assigned to the `$LetterGrade` variable)

```
$LetterGrade = "B";

preg_match("[A-DF]", $LetterGrade);// returns 1
```

DEFINING CHARACTER CLASSES

- The `^` metacharacter (placed immediately after the opening bracket of a character class) specifies optional characters to exclude in a pattern match
  (the following example excludes the letter `E` and `G-Z` from an acceptable pattern match in the `$LetterGrade` variable)

```
$LetterGrade = "A";
preg_match("[^EG-Z]", $LetterGrade);
                                    //returns 1
$LetterGrade = "E";
preg_match("[^EG-Z]", $LetterGrade);
                                    //returns 0
```

DEFINING CHARACTER CLASSES – PCRE CHARACTER TYPES

- Also, there are special characters that can be used to represent different types of data

```
preg_match("/^[\w-]+(\.[\w-]+)*@[\w-]+
        (\.[\w-]+)*(\.[a-zA-Z]{2,})$/", $Email);
```

Escape Sequence	Description
\a	alarm (hex 07)
\cx	"control-x", where x is any character
\d	any decimal digit
\D	any character not in \d
\e	escape (hex 1B)
\f	formfeed (hex 0C)
\h	any horizontal whitespace character
\H	any character not in \h
\n	newline (hex 0A)
\r	carriage return (hex 0D)
\s	any whitespace character
\S	any character not in \s
\t	tab (hex 09)
\v	any vertical whitespace character
\V	any character not in \v
\w	any letter, number, or underscore character
\W	any character not in \w

MATCHING MULTIPLE PATTERN CHOICES

- The | metacharacter is used to specify an alternate set of patterns
  - The | metacharacter is essentially the same as using the OR operator to perform multiple evaluations in a conditional expression

```
preg_match("/\.(com|org|net)$/i",
        "http://www.education.gov"); // returns 0

preg_match("/\.(com|org|net)$/i",
         "http://www.education.com"); // returns 1
```

PATTERN MODIFIERS

- **Pattern modifiers** are letters placed after the closing delimiter that change the default rules for interpreting matches
  - The pattern modifier, `i`, indicates that the case of the letter does not matter when searching
  - The pattern modifier, `m`, allows searches across newline characters
  - The pattern modifier, `s`, changes how the . (period) metacharacter works

AUTOGLOBALS (SUPER GLOBAL VARIABLES)

- **Autoglobals** are predefined variables - global (associative) arrays that are useful for getting information about the runtime environment, the state of PHP or the user
- $_SERVER - contains information about headers, paths, and script locations
- $_POST - contains values from a user form submitted with the "post" method
- $_GET - contains values from a user form submitted with the "get" method

AUTOGLOBALS

- $_COOKIE - contains values passed to the current script as HTTP cookie
- $_SESSION - contains variables that are available to the current script
- $_FILES - contains information about uploaded files
- $GLOBALS – contains references to all variables that are defined with global scope

WEB FORMS

- **Web forms** are providing effective way to collect data
  - allow users to enter and submit data to a processing script
- A Web form is a standard HTML form with two required attributes in the opening `<form>` tag:
  - **Action attribute:** Identifies the program on the Web server that will process the form data when it is submitted
  - **Method attribute:** Specifies how the form data will be sent to the processing script

`METHOD` ATTRIBUTE

- `method` attribute with value
  - "post" - embeds the form data in the request message; PHP automatically creates and populates a `$_POST` array
  - "get" - appends the form data to the URL specified in the form's action attribute; PHP creates and populates a `$_GET` array

METHOD ATTRIBUTE

- Form fields are sent to the Web server as a *name/value* pair
  - The *name* is the key of an element, while the *value* (data that the user enters in the input control) is the value of that element in the `$_POST` or `$_GET` array
- When submitting data using the "get" method, form data is appended to the URL specified by the action attribute
  - Name/value pairs appended to the URL are called **URL tokens**

PROCESSING FORM DATA

- When a user clicks the submit button, the form data is sent to the server for processing
- A **form handler** is a script/ program that processes the form data
- It performs the following:
  - Validates form data
  - Process the submitted data
  - Returns appropriate output

RECEIVING FORM DATA

- In PHP the form data is accessed using the `$_POST/$_GET` array

```
$name = $_POST['name'];

$email = $_POST['email'];

----

$name = $_GET['name'];

$email = $_GET['email'];
```

Example: ScholarshipForm.html, process_scholarsihip.php

FORM DATA VALIDATION

- It is necessary to validate form data to ensure that the data matches the expected type and format
- PHP functions that can be used for data validation
  - `empty()` - used to determine if a variable contains a value (returns FALSE if the variable has a nonempty and nonzero value, TRUE if the variable has an empty or zero value)
  - `filter_var()` - filter and validate data
    - takes two arguments: the value to be filtered and the type to be applied
  - `filter_var_array()` - applies the same or different filters to multiple values at once
- Validation filters
  - FILTER_VALIDATE_EMAIL, FILTER_VALIDATE_INT, FILTER_VALIDATE_FLOAT, …

WEB FORM EXAMPLE

- *ScholarshipForm.html*
- *process_scholarsihip1.php*

ALL-IN-ONE FORM

- A **two-part form** has one script that displays the form and another script that processes the form data
- For simple forms that require only minimal processing, can be used **All-in-One form** – the same script display the form and process form data
- When the user click the submit button, the script submit data to the current script

ALL-IN-ONE FORM

- `isset()` - determine if the variable has been set (the form is submitted)
  - input argument is the name assigned to the Submit button in the Web form

```
if (isset ($_POST['Submit'])) {
  // Process the data
}
else {
  // Display the Web form
}
```

ALL-IN-ONE FORM EXAMPLE

- *Example3-53.php*

DISPLAYING DYNAMIC CONTENT

- By passing URL tokens to a PHP script, many different information can be passed to the script
- One very simple example is a Web page template
  - using static and dynamic content sections, a single PHP script can produce different outputs

DISPLAYING DYNAMIC CONTENT

- A **Web template** is a single Web page that is divided into separate sections
- The contents of the individual sections can be placed in separate files
- The navigation can be implemented using hyperlinks and buttons

```
<a href = "index.php?content=Home">Home</a>

<input type="submit" name="content"
                                    value="Home" />
```

DISPLAYING DYNAMIC CONTENT EXAMPLE